

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SHORTING



NAMA: Rachmadias Vebriansyah Aflah Islami

NIM: 109082500004

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Algoritma pengurutan (*sorting*) merupakan salah satu konsep fundamental dalam pemrograman untuk menyusun data secara teratur. Dua metode yang sering dipelajari adalah *Selection Sort* dan *Insertion Sort*. *Selection Sort* bekerja dengan cara memindai elemen data untuk mencari nilai ekstrem (terkecil atau terbesar), kemudian menukarnya dengan elemen di posisi ujung rentang data yang belum terurut. Proses seleksi dan pertukaran ini diulangi terus-menerus hingga menyisakan satu elemen saja.

Di sisi lain, *Insertion Sort* menggunakan pendekatan yang berbeda dengan menyisipkan elemen pada posisi yang tepat layaknya menyusun kartu. Algoritma ini mengambil sebuah data, lalu membandingkannya secara berurutan (*sequential search*) dengan data-data yang sudah terurut di sebelah kirinya. Selama data di sebelah kirinya tidak memenuhi kondisi urutan, data tersebut akan digeser ke kanan untuk membuka ruang kosong bagi elemen baru. Kedua algoritma ini sangat efektif dipelajari untuk memahami logika manipulasi indeks array serta struktur perulangan bersarang.

B. Guided

1. [insertionSort]

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    /* I.S. terdefinisi array T yang berisi n bilangan bulat
       F.S. array T terurut secara mengecil atau descending dengan
       INSERTION SORT */

    var temp, i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
```

```

        j = j - 1
    }

    T[j] = temp
    i = i + 1
}
}

func main() {
    var data arrInt
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSort(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS C:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\109082500004_Rachmadi-az-Vebriansyah> go run "c:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\109082500004_Rachmadi-az-Vebriansyah\Pertemuan-14Modul14\Guided\nol.go"
5
4 45 7 23
34 5 3
45 34 23 7 4

```

Penjelasan :

Program ini menggunakan algoritma Insertion Sort untuk mengurutkan sekumpulan angka secara descending (menurun). Logikanya mirip seperti menyusun kartu di tangan: program mengambil satu angka (disebut temp), lalu membandingkannya dengan angka-angka di sebelah kirinya. Jika angka di sebelah kiri lebih kecil dari temp, maka angka tersebut digeser ke kanan untuk memberi ruang bagi temp agar bisa masuk ke posisi yang lebih depan.

Proses ini dilakukan berulang kali mulai dari indeks kedua hingga indeks terakhir. Karena syarat pada loop dalam adalah $temp > T[j-1]$, maka angka

yang lebih besar akan selalu didorong ke arah kiri (indeks awal). Hasil akhirnya adalah array yang tertata rapi dari nilai terbesar ke nilai terkecil, yang kemudian dicetak ke layar menggunakan spasi sebagai pemisah.

2. [insertionSortString]

```
package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrString
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSortString(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}
```

Output :

```
PS C:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\109082500004 Rachmadi  
● az-Vebriansyah> go run "c:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\  
109082500004_Rachmadi\az-Vebriansyah\Pertemuan-14Modul14\Guided\no2.go"  
5  
budi andi cici zaki dedi  
andi budi cici dedi zaki
```

Penjelasan :

Program ini mengimplementasikan algoritma Insertion Sort untuk mengurutkan sekumpulan data teks atau string secara ascending (secara alfabetis dari A sampai Z). Pada dasarnya, cara kerjanya mirip dengan menyisipkan kartu ke posisi yang tepat. Program akan mengambil satu elemen teks sebagai target penyimpanan sementara (temp), lalu membandingkannya dengan teks-teks yang berada di sebelah kirinya.

Proses perbandingan ini memanfaatkan operasi logika $temp < T[j-1]$. Di dalam bahasa Go, operator perbandingan pada string bekerja berdasarkan urutan leksikografis (kamus atau nilai ASCII). Jika teks di sebelah kiri memiliki urutan abjad yang lebih besar daripada teks di dalam temp, maka teks di kiri tersebut akan digeser ke kanan. Proses penggeseran ini terus berlanjut hingga program menemukan posisi abjad yang tepat untuk menyisipkan teks temp tersebut, menghasilkan susunan kata yang rapi dari A hingga Z.

3. [SelSortArray]

```
package main

import "fmt"

func selecetionSortArray(angka *[5]int){
    var idx_min, i, j int
    for i = 0; i < len(angka) - 1; i++ { //loop luar (iterasi
        sortingnya)
        idx_min = i
        for j = i + 1; j < len(angka); j++ { //loop dalam (mencari
            nilai ekstrim)
            if angka[j] <= angka[idx_min] { //sorting terkecil ke
                terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            angka[i], angka[idx_min] = angka[idx_min], angka[i]
        }
    }
}

func main(){
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORITNG ===")
    selecetionSortArray(&arrAngka)
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}
```

Output :

```
PS C:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\109082500004_Rachmadi
az-Vebriansyah> go run "c:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\
109082500004_Rachmadi\az-Vebriansyah\Pertemuan-14Modul14\Guided\no3.go"
Masukkan data angka ke-0 : 25
Masukkan data angka ke-1 : 43
Masukkan data angka ke-2 : 45
Masukkan data angka ke-3 : 1
Masukkan data angka ke-4 : 21

=== SEBELUM DISORTING ===
25 - 43 - 45 - 1 - 21 -
=== SETELAH DISORTING ===
1 - 21 - 25 - 43 - 45 -
PS C:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\109082500004_Rachmadi
```

Penjelasan :

Program ini mengimplementasikan algoritma Selection Sort untuk mengurutkan 5 elemen angka di dalam array secara ascending (dari terkecil ke terbesar).

Cara kerja algoritma ini adalah dengan memisahkan array menjadi dua bagian, yaitu bagian yang sudah terurut dan bagian yang belum terurut.

Program akan memindai bagian yang belum terurut untuk mencari angka dengan nilai paling kecil melalui variabel pencatat indeks `idx_min`.

Proses pencarian nilai terkecil ini dilakukan oleh *loop* dalam (`j`). Setelah mengitari sisa elemen array dan berhasil menemukan angka terkecil, program akan memeriksa apakah posisi angka terkecil tersebut berbeda dengan posisi awal iterasi (`i`). Jika berbeda, program akan menukar (*swap*) angka terkecil tersebut ke posisi `i` menggunakan fitur *multiple assignment* khas Go (`angka[i], angka[idx_min] = angka[idx_min], angka[i]`). Langkah ini terus diulangi hingga seluruh array berhasil tersusun dengan benar dari angka terkecil hingga terbesar.

4. [SelSortStruc]

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK       float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { //loop luar
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ { //loop dalam mencari nilai
            ektrim
            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting terkecil ke
                terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println(" === SEBELUM SORITNG === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
```

```

        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println(" === SETELAH SORITNG === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```

=====
=== SETELAH SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : acca
NIM : 123
IPK : 2.00
--- Data Mahasiswa Ke-1 ---
Nama : oyan
NIM : 758
IPK : 2.98
--- Data Mahasiswa Ke-2 ---
Nama : dwi
NIM : 239
IPK : 3.60
--- Data Mahasiswa Ke-3 ---
Nama : iyanz
NIM : 743
IPK : 3.99
--- Data Mahasiswa Ke-4 ---
Nama : qya
NIM : 1090
IPK : 4.00
=====

```

Penjelasan :

Program ini menerapkan algoritma Selection Sort untuk mengurutkan data bertipe gabungan (*struct*) bernama mahasiswa. Objek yang diurutkan terdiri dari kombinasi data nama, NIM, dan IPK untuk 5 orang mahasiswa. Fokus utama pengurutan ini didasarkan pada nilai IPK yang diurutkan secara

ascending, artinya program akan menyusun data mahasiswa mulai dari yang memiliki IPK paling rendah hingga IPK paling tinggi.

Saat proses pengurutan berlangsung di fungsi `SelectionSortArray`, program akan menyisir array untuk mencari nilai IPK terkecil melalui *loop* dalam, lalu mencatat posisinya di variabel `idx_min`. Ketika elemen dengan IPK terkecil pada iterasi tersebut ditemukan, program tidak hanya memindahkan nilai IPK-nya saja, melainkan menukar satu kesatuan data objek mahasiswa (nama, NIM, dan IPK sekaligus) ke posisi depan menggunakan operasi penukaran langsung khas bahasa Go. Hasil akhirnya, seluruh identitas mahasiswa ikut berpindah tempat secara utuh mengikuti urutan nilai IPK mereka.

C. Unguided

1. Soal Unguided 1

THercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort. Masukan dimulai dengan sebuah integer n ($0 < n < 1000$) banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat. Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

Jawaban :

```
package main

import "fmt"

func selectionSortAsc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
        if idxMin != i {
            arr[i], arr[idxMin] = arr[idxMin], arr[i]
        }
    }
}

func main() {
    var n, m int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&m)
        rumah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        selectionSortAsc(rumah, m)

        for j := 0; j < m; j++ {
```

```

        fmt.Print(rumah[j], " ")
    }
    fmt.Println()
}

```

Output :

```

PS C:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\109082500004_Rachmadi
● az-Vebriansyah> go run "c:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\
109082500004_Rachmadi\az-Vebriansyah\Pertemuan-14Modul14\Unguided\no-1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 12 8 2
2 8 12

```

Penjelasan :

program menggunakan algoritma *selection sort* untuk mengurutkan nomor rumah kerabat Hercules dari yang terkecil ke terbesar. Data yang dimasukkan terdiri dari beberapa daerah, lalu setiap daerah diproses satu per satu hingga menghasilkan urutan angka yang rapi. Penggunaan metode ini membantu program menemukan nilai terkecil secara bertahap sampai seluruh data terurut.

Selain itu, soal ini menunjukkan bahwa algoritma sorting sangat penting dalam pengolahan data. Dengan data yang sudah terurut, informasi menjadi lebih mudah dibaca dan dipahami. Program juga melatih penggunaan array dan perulangan dalam menyimpan serta memproses banyak data sekaligus.

2. Soal Unguided 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil. Format Masukan masih persis sama seperti sebelumnya. Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

Jawaban :

```
package main

import "fmt"

func selectionSortAsc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
        if idxMin != i {
            arr[i], arr[idxMin] = arr[idxMin],
arr[i]
        }
    }
}

func selectionSortDesc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        idxMax := i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[idxMax] {
                idxMax = j
            }
        }
        if idxMax != i {
            arr[i], arr[idxMax] = arr[idxMax],
arr[i]
        }
    }
}
```

```

    }

    func main() {
        var n, m, val int
        fmt.Scan(&n)

        for i := 0; i < n; i++ {
            fmt.Scan(&m)
            var ganjil []int
            var genap []int

            for j := 0; j < m; j++ {
                fmt.Scan(&val)
                if val%2 != 0 {
                    ganjil = append(ganjil, val)
                } else {
                    genap = append(genap, val)
                }
            }

            selectionSortAsc(ganjil)
            selectionSortDesc(genap)

            for _, v := range ganjil {
                fmt.Print(v, " ")
            }
            for _, v := range genap {
                fmt.Print(v, " ")
            }
            fmt.Println()
        }
    }
}

```

Output :

```

PS C:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\109082500004_Rachmadi
az-Vebriansyah> go run "c:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\
109082500004_Rachmadi\az-Vebriansyah\Pertemuan-14Modul14\Unguided\no-2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
4 9 1 12 8
1 9 12 8
PS C:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\109082500004_Rachmadi
az-Vebriansyah>

```

Penjelasan :

program tidak hanya mengurutkan data, tetapi juga memisahkan nomor rumah ganjil dan genap. Nomor ganjil dicetak lebih dahulu secara menaik, sedangkan nomor genap dicetak secara menurun. Cara ini dibuat agar Hercules dapat mengurangi frekuensi menyeberang jalan saat mengunjungi rumah kerabatnya.

Soal ini mengajarkan bahwa algoritma dapat disesuaikan dengan kebutuhan tertentu, tidak hanya sekadar mengurutkan data biasa. Selain penggunaan sorting, program juga memanfaatkan proses seleksi data berdasarkan kondisi ganjil dan genap sehingga hasil keluaran menjadi lebih spesifik sesuai aturan yang diminta.

3. Soal Unguided 3

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya. Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array. Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap"

Jawaban :

```
package main

import "fmt"

func insertionSortAsc(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        temp := arr[i]
        j := i
        for j > 0 && temp < arr[j-1] {
            arr[j] = arr[j-1]
            j--
        }
        arr[j] = temp
    }
}

func main() {
    var val int
    var data []int

    for {
        fmt.Scan(&val)
        if val < 0 {
            break
        }
        data = append(data, val)
    }

    if len(data) == 0 {
        return
    }

    insertionSortAsc(data)
```

```

for i := 0; i < len(data); i++ {
    fmt.Print(data[i], " ")
}
fmt.Println()

if len(data) < 2 {
    fmt.Println("Data berjarak 0")
    return
}

diff := data[1] - data[0]
isConstant := true
for i := 1; i < len(data)-1; i++ {
    if (data[i+1] - data[i]) != diff {
        isConstant = false
        break
    }
}

if isConstant {
    fmt.Printf("Data berjarak %d\n", diff)
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}

```

Output :

```

PS C:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\109082500004_Rachmadi
az-Vebriansyah> go run "c:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\
109082500004_Rachmadi\az-Vebriansyah\Pertemuan-14Modul14\Unguided\3.go"
43 37 31 25 19 13 7 1 -1
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\Lenovo\OneDrive\Documents\MATA KULIAH IF-13-06\MATKUL 13-06 Semester 2\Laporan Praktikum\109082500004_Rachmadi
az-Vebriansyah>

```

Penjelasan :

program menggunakan metode *insertion sort* untuk mengurutkan bilangan yang dimasukkan pengguna. Setelah data terurut, program memeriksa apakah selisih antar bilangan selalu sama atau tidak. Jika sama, maka data disebut memiliki jarak tetap.

Dari soal ini dapat dipahami bahwa proses sorting dapat membantu analisis data lebih lanjut. Dengan data yang sudah tersusun rapi, program lebih mudah menentukan pola atau hubungan antar angka. Selain itu, soal ini juga melatih penggunaan array, perulangan, dan pengecekan kondisi dalam pemrograman.

4. Soal Unguided 4

[isi dengan soal unguided 3]

Jawaban :

```
package main

import "fmt"

const nMax int = 7919 [cite: 253, 254]

type Buku struct { [cite: 255]
    id, judul, penulis, penerbit string [cite: 256]
    eksemplar, tahun, rating      int      [cite: 256, 257]
}

type DaftarBuku [nMax + 1]Buku // Menggunakan 1..nMax sesuai
    spesifikasi soal [cite: 258]

func DaftarkanBuku(pustaka *DaftarBuku, n int) { [cite: 266]
    // I.S. sejumlah n data buku telah siap pada piranti
    masukan [cite: 266]
    // F.S. pustaka berisi sejumlah n data buku [cite: 267]
    for i := 1; i <= n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
            &pustaka[i].penulis, &pustaka[i].penerbit,
            &pustaka[i].eksemplar, &pustaka[i].tahun,
            &pustaka[i].rating)
    }
}

func UrutBuku(pustaka *DaftarBuku, n int) { [cite: 269, 270]
    // I.S. Array pustaka berisi n data buku [cite: 271]
    // F.S. Array pustaka terurut menurun/mengecil terhadap
    rating menggunakan Insertion Sort [cite: 272]
    for i := 2; i <= n; i++ {
        temp := pustaka[i]
        j := i
        for j > 1 && temp.rating > pustaka[j-1].rating {
            pustaka[j] = pustaka[j-1]
            j--
        }
        pustaka[j] = temp
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) { [cite: 268]
    // I.S. array pustaka berisi n buah data buku dan belum
    terurut [cite: 268]
```

```

// F.S. Tampilan data buku terfavorit (rating tertinggi)
[cite: 268]
if n == 0 {
    return
}
fav := pustaka[1]
for i := 2; i <= n; i++ {
    if pustaka[i].rating > fav.rating {
        fav = pustaka[i]
    }
}
fmt.Printf("%s, %s, %s, %d\n", fav.judul, fav.penulis,
    fav.penerbit, fav.tahun) [cite: 268]
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) { [cite: 273,
    274]
    // I.S. pustaka berisi n data buku yang sudah terurut
    menurut rating [cite: 275]
    // F.S. Laporan 5 judul buku dengan rating tertinggi
    [cite: 275]
    limit := 5
    if n < 5 {
        limit = n [cite: 275]
    }
    for i := 1; i <= limit; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) { [cite: 277]
    // I.S. pustaka berisi n data buku yang sudah terurut
    menurut rating [cite: 277]
    // F.S. Laporan salah satu buku dengan rating r memakai
    Binary Search [cite: 277]
    left := 1
    right := n
    foundIdx := -1

    for left <= right {
        mid := (left + right) / 2
        if pustaka[mid].rating == r {
            foundIdx = mid
            break
        } else if pustaka[mid].rating < r { // Karena terurut
            menurun, rating besar ada di kiri
            right = mid - 1
        } else {
            left = mid + 1
        }
    }
}

```

```

        if foundIdx != -1 {
            b := pustaka[foundIdx]
            fmt.Printf("%s, %s, %s, %d, %d, %d\n", b.judul,
                b.penulis, b.penerbit, b.tahun, b.eksemplar, b.rating)
            [cite: 277]
        } else {
            fmt.Println("Tidak ada buku dengan rating seperti
                itu") [cite: 277]
        }
    }

func main() {
    var pustaka DaftarBuku
    var n, cariRating int

    fmt.Scan(&n) [cite: 260]
    DaftarkanBuku(&pustaka, n)
    fmt.Scan(&cariRating) [cite: 262]

    fmt.Println("\n=== OUTPUT ===")
    CetakTerfavorit(pustaka, n)

    UrutBuku(&pustaka, n)
    Cetak5Terbaru(pustaka, n)

    CariBuku(pustaka, n, cariRating)
}

```

Output :

ERROR PUYENG.

Penjelasan :

program perpustakaan menggunakan struktur data *struct* dan array untuk menyimpan informasi buku. Setiap buku memiliki beberapa atribut seperti judul, penulis, penerbit, tahun, dan rating. Dengan menggunakan struct, data yang kompleks dapat disimpan dengan lebih terorganisir.

Selain itu, program menunjukkan pentingnya pengelolaan data dalam jumlah besar. Array digunakan untuk menyimpan banyak data buku, sedangkan struct membantu mengelompokkan informasi yang saling berkaitan. Konsep ini sangat berguna dalam pembuatan aplikasi nyata seperti sistem perpustakaan atau database sederhana.

D. Kesimpulan

pemilihan algoritma pengurutan yang tepat sangat memengaruhi bagaimana struktur data di dalam program dimanipulasi. Selection Sort terbukti andal ketika kita ingin meminimalkan jumlah operasi pertukaran (*swap*) data, karena pertukaran hanya terjadi satu kali pada setiap putaran luar setelah posisi ekstrem ditemukan. Algoritma ini sangat ideal diaplikasikan pada pemisahan kategori data spesifik secara simultan seperti pengelompokan nomor rumah ganjil dan genap.

Sementara itu, Insertion Sort memiliki keunggulan performa yang sangat baik ketika menangani data yang datang secara bertahap atau data yang sudah hampir terurut. Melalui proses penyisipan langsung, algoritma ini mampu menjaga kestabilan data lama, seperti yang diterapkan pada penghitungan *median* berkala maupun manajemen urutan *rating* buku di perpustakaan. Pemahaman menyeluruh terhadap kondisi penghentian perulangan (baik secara *ascending* maupun *descending*) menjadi kunci utama keberhasilan implementasi kedua algoritma pengurutan ini.